==========

-- Total Sales across each category

category- sum(sales)

Electronics- 20000
Furniture- 30000
Clothing- 60000
Toys- 10000
Books- 5000

```
SELECT DISTINCT category from sales;

SELECT category, SUM(amount) as total_sales
FROM sales
GROUP BY category;
```

-- this will work but might not make much sense

```
SELECT SUM(amount) as total_sales
FROM sales
GROUP BY category;
```

-- SELECT category, sale_date, SUM(amount) as total_sales FROM sales GROUP BY category;

category- sum(amount)
Electronics- 20000
Furniture- 30000
Clothing- 60000
Toys- 10000
Books- 5000

electronics- 2500 records

```
-- the below query will work
SELECT category, max(sale_date), SUM(amount) as total_sales
FROM sales
GROUP BY category;

-- this is equivalent to distinct
SELECT category
FROM sales
GROUP BY category;

SELECT category , product, SUM(amount) as total_sales
FROM sales
GROUP BY category;
```

-- count of products in each category

```
SELECT category, count(distinct product) as total_products
FROM sales
GROUP BY category;
```

-- total quantity sold for each product

```
SELECT product,
count(*) as total_orders ,
SUM(quantity) as total_quantity
FROM sales
GROUP BY product;
```

-- 3 products with maximum sales volume

```
SELECT product, SUM(quantity) as total_quantity
FROM sales
GROUP BY product
ORDER BY total_quantity desc
limit 3;
```

-- total sales and number of transactions per store location

```
SELECT store_location,
SUM(amount) as total_sales, count(*) as num_transactions
FROM sales
GROUP BY store_location;
```

-- average sale amount by each customer

```
SELECT customer_id, AVG(amount) as average_sale
FROM sales
GROUP BY customer_id;
```

```
SELECT customer_id, AVG(amount) as average_sale,
SUM(amount) as total_purchase_amount,
count(*) as num_of_orders,
SUM(amount)/count(*) as avgsale
FROM sales
GROUP BY customer_id;
```

id	amount
1	10000
2	15000
3	null
4	5000

total(amount) = 30000
max(amount) = 15000
avg(amount) = 10000

sum(amount) / count(*)

-- Total sales by month and category

how many rows will be there in output

6 months

5 categories

30 rows will be there in output

Electronics 2023-01
Electronics 2023-02
.
.
.
.

```
SELECT DATE_FORMAT(sale_date,'%Y-%m') as sale_month, category,
SUM(amount) as total_sales
FROM sales
GROUP BY sale_month, category
```

-- TOTAL SALES BY STORE LOCATION WHERE TOTAL SALES EXCEED 1 MILLION

```
SELECT store_location, SUM(amount) as total_sales
FROM sales
GROUP BY store_location
HAVING total_sales > 1000000;
```

5 groups

now we want to show only those groups where total_sales > 1 million

2 groups matched the criteria

-- I want to find highest order amount in each category

```
SELECT category, MAX(amount) as highest_sale
FROM sales
GROUP BY category;
```

-- TOTAL SALES AMOUNT AND AVG QUANTITY SOLD PER PRODUCT IN NEW YORK

```
SELECT product, SUM(amount) as total_sales,
AVG(quantity) as avg_quantity
FROM sales
WHERE store_location = 'New York'
GROUP BY product;
```

-- WE JUST NEED TO DO THE ANALYSIS ON THE PRODUCT 'TABLET'

```
SELECT product, SUM(amount) as total_sales,
sum(quantity) as sum_quantity
FROM sales
WHERE store_location = 'New York'
GROUP BY product
HAVING product = 'Tablet';
```

37 products

37 groups

tablet

```
SELECT product, SUM(amount) as total_sales,
sum(quantity) as sum_quantity
FROM sales
WHERE store_location = 'New York' AND product = 'Tablet'
GROUP BY product;
```

40000 ROWS

5 groups based on city

city	count_of_orders
Hyderabad	8000
chennai	10000
bangalore	6000
pune	5000
delhi	11000

lets say we apply filter based on january month

city	count_of_orders
hyderabad	2000
chennai	3000
bangalore	1500
pune	1000
delhi	3500

it makes sense to apply having clause if we want to either filter based on count_of_orders or any other aggregations like total_sales.

filters on aggregation is not possible in where clause.

-- total sales and total quantity sold each month

```
SELECT DATE_FORMAT(sale_date,'%Y-%m') as sale_month,
SUM(amount) as total_sales,
SUM(quantity) as total_quantity
FROM sales
GROUP BY sale_month;
```

-- from each store I want the count of unique customers

```
SELECT store_location,
COUNT(DISTINCT customer_id) as customer_count
FROM sales
GROUP BY store_location;
```

-- monthly sales by each customer

```
SELECT DATE_FORMAT(sale_date,'%Y-%m') as sale_month, customer_id,
SUM(amount) as total_sales
FROM sales
GROUP BY sale_month, customer_id
```

-- monthly sales by each customer who have placed atleast 8 orders

```
SELECT DATE_FORMAT(sale_date,'%Y-%m') as sale_month, customer_id,
SUM(amount) as total_sales
FROM sales
GROUP BY sale_month, customer_id
HAVING count(*) >= 8;
```

-- monthly sales by premium customers

```
SELECT DATE_FORMAT(sale_date,'%Y-%m') as sale_month, customer_id,
SUM(amount) as total_sales
FROM sales
GROUP BY sale_month, customer_id
HAVING avg(amount) > 600;
```

-- total sales amount by category with sales in jan 2023

```
SELECT category, SUM(amount) as total_sales
FROM sales
WHERE MONTH(sale_date) = 1 AND YEAR(sale_date) = 2023
GROUP BY category;
```

```
-- TOTAL SALES AMOUNT BY PREMIUM CUSTOMER(>7000)
 I JUST WANT TO CONSIDER SALES ON WEEKDAYS

SELECT customer_id, SUM(amount) as total_sales
FROM sales
WHERE DAYOFWEEK(sale_date) BETWEEN 2 AND 6
GROUP BY customer_id
HAVING total_sales > 7000;

-- TOTAL SALES AND NUMBER OF ORDERS PER STORE LOCATION WITH SALES HAPPENING IN THE
MONTH OF JAN 2023, ORDERED BY NUMBER OF TRANSACTIONS

SELECT store_location, SUM(amount) as total_sales, count(*) as num_transactions
FROM sales
WHERE sale_date BETWEEN '2023-01-01' AND '2023-01-31'
GROUP BY store_location
ORDER BY num_transactions desc;
```